Funciones en PHP

Las funciones en PHP son una herramienta poderosa que nos permiten encapsular bloques de código que realizan una tarea específica y reutilizarlos en nuestro programa. Son bloques de código que pueden ser llamados desde diferentes partes del programa y que nos permiten mejorar la estructura y legibilidad del mismo.

Para crear una función en PHP, utilizamos la siguiente sintaxis:

```
function nombre_de_la_funcion($parametro1, $parametro2, ...) {
  // código a ejecutar
  return $valor_de_retorno;
}
```

Aquí estamos definiendo una función llamada **nombre_de_la_funcion** que recibe uno o varios parámetros (que se especifican entre paréntesis) y que ejecuta un bloque de código cuando es llamada. La función puede o no retornar un valor utilizando la sentencia **return**.

Por ejemplo, podríamos definir una función que calcule el área de un círculo utilizando el valor de su radio:

```
function calcular_area_circulo($radio) {
  $area = pi() * $radio * $radio;
  return $area;
}
```

En este caso, estamos definiendo una función llamada **calcular_area_circulo** que recibe un parámetro **$radio** y que utiliza la fórmula del área del círculo para calcular su valor. La función retorna el valor del área calculada.

Para llamar a una función en PHP, simplemente utilizamos su nombre seguido de los parámetros que recibe, separados por comas. Por ejemplo:

```
$area = calcular_area_circulo(5);
echo "El área del círculo es: " . $area;
```

En este caso, estamos llamando a la función **calcular_area_circulo** con un parámetro de valor **5**. La función devuelve el valor del área calculada y lo asignamos a la variable **$area**. Luego, imprimimos un mensaje que incluye el valor del área calculada.

Además de recibir parámetros, las funciones también pueden definir variables internas que sólo existen dentro del bloque de código de la función. Estas variables se llaman variables locales y se definen utilizando la palabra clave **var** o **global**, según si queremos que la variable sea local o global al programa.

Las variables locales y globales:

Las variables locales y globales son conceptos importantes en PHP y en muchos otros lenguajes de programación.

Las variables locales son aquellas que se definen dentro del bloque de código de una función y que *sólo existen mientras la función se está ejecutando*. Esto significa que una vez que la función ha terminado de ejecutarse, las variables locales desaparecen y no pueden ser accedidas desde otras partes del programa.

Por ejemplo, en la función **calcular_area_circulo** que definimos antes, la variable **$area** es una variable local. Sólo existe dentro del bloque de código de la función y desaparece una vez que la función ha terminado de ejecutarse.

Las variables globales, por otro lado, son aquellas que se definen fuera de cualquier función o bloque de código y que pueden ser accedidas desde cualquier parte del programa, incluyendo dentro de las funciones.

Por ejemplo, supongamos que tenemos una variable global llamada $pi que contiene el valor de Pi:

```
$pi = 3.14159;
```

Podríamos utilizar esta variable dentro de la función calcular_area_circulo para calcular el área del círculo de la siguiente manera:

```
function calcular_area_circulo($radio) {
  global $pi; // indicamos que la variable $pi es global
  $area = $pi * $radio * $radio;
  return $area;
}
```

Aquí estamos utilizando la palabra clave global para indicar que la variable **$pi** es global y que queremos acceder a su valor dentro de la función. Luego, utilizamos el valor de **$pi** en el cálculo del área del círculo.

Es importante tener en cuenta que el uso de variables globales debe ser manejado con precaución, ya que pueden dificultar la comprensión y el mantenimiento del código. En general, se recomienda utilizar variables locales siempre que sea posible y evitar el uso de variables globales a menos que sea estrictamente necesario.

Las variables locales son aquellas que se definen dentro del bloque de código de una función y que sólo existen mientras la función se está ejecutando. Las variables globales, por otro lado, son aquellas que se definen fuera de cualquier función o bloque de código y que pueden ser accedidas desde cualquier parte del programa. Es importante tener en cuenta que el uso de variables globales debe ser manejado con precaución y que se recomienda utilizar variables locales siempre que sea posible.

Algunas Variables Globales predefinidas

PHP tiene varias variables globales predefinidas que pueden ser útiles en diferentes situaciones.
Ejemplos

$_GET: Esta variable global contiene los parámetros de una petición HTTP enviada utilizando el método GET. Por ejemplo, si tienes una URL como *http://ejemplo.com/index.php?nombre=Juan&edad=30*, puedes acceder a los valores de los parámetros nombre y edad de la siguiente manera:

```
$nombre = $_GET['nombre'];
$edad = $_GET['edad'];
```

$_POST: Esta variable global contiene los datos de una petición HTTP enviada utilizando el método POST. Por ejemplo, si tienes un formulario HTML que envía datos a un script PHP utilizando el método POST, puedes acceder a los valores de los campos del formulario de la siguiente manera:

```
$nombre = $_POST['nombre'];
$apellido = $_POST['apellido'];
```

$_SESSION: Esta variable global se utiliza para almacenar datos entre diferentes peticiones HTTP realizadas por el mismo usuario. Por ejemplo, puedes utilizar esta variable para almacenar el nombre de usuario de un usuario que ha iniciado sesión en tu sitio web:

```
session_start(); // iniciamos la sesión
$_SESSION['nombre_usuario'] = 'Juan'; // almacenamos el nombre de usuario en la sesión
```

Luego, puedes acceder al valor de la variable $_SESSION['nombre_usuario'] en cualquier parte de tu código mientras la sesión esté activa.

$_SERVER: Esta variable global contiene información sobre el servidor web y la petición HTTP actual. Por ejemplo, puedes acceder a la dirección IP del cliente que ha realizado la petición utilizando la siguiente línea de código:

```
$ip_cliente = $_SERVER['REMOTE_ADDR'];
```

Estos son sólo algunos ejemplos de las variables globales predefinidas que existen en PHP.

Método GET

El método GET es uno de los métodos de solicitud HTTP que se utiliza para enviar información a un servidor web. Es comúnmente utilizado para solicitar recursos (como páginas web) y para enviar datos a través de formularios HTML.

Cuando utilizas el método GET, los datos se envían como parte de la URL de la petición HTTP. Es decir, los parámetros y sus valores se incluyen directamente en la URL separados por el carácter "?" y por el carácter "&" si hay varios parámetros. Por ejemplo, si tienes un formulario HTML que envía dos parámetros (nombre y apellido) utilizando el método GET, la URL resultante podría ser algo como esto:

```
http://ejemplo.com/procesar_formulario.php?nombre=Juan&apellido=Pérez
```

En este ejemplo, http://ejemplo.com/procesar_formulario.php es la dirección URL del script PHP que procesará los datos enviados por el formulario, nombre es el nombre del primer parámetro y Juan es el valor del primer parámetro, apellido es el nombre del segundo parámetro y Pérez es el valor del segundo parámetro.

Cuando el servidor recibe una petición HTTP utilizando el método GET, los datos se pueden acceder desde el script PHP utilizando la variable global $_GET. Esta variable es un arreglo asociativo que contiene los nombres de los parámetros como claves y los valores de los parámetros como valores. Por ejemplo, para acceder a los valores de los parámetros nombre y apellido en el ejemplo anterior, podrías hacer lo siguiente:

```
$nombre = $_GET['nombre'];
$apellido = $_GET['apellido'];
```

Es importante tener en cuenta que el método GET no es adecuado para enviar datos sensibles (como contraseñas) ya que los datos se incluyen directamente en la URL y pueden ser vistos por cualquier persona que tenga acceso a la URL (por ejemplo, en un registro de actividad del servidor). En esos casos, se recomienda utilizar el método POST, que envía los datos en el cuerpo de la solicitud HTTP en lugar de incluirlos en la URL.

El método GET se recomienda cuando se quiere solicitar recursos o información del servidor y cuando los datos enviados no son sensibles o no necesitan ser protegidos. Algunas situaciones comunes en las que se utiliza el método GET son:

- Acceder a páginas web: Cuando se accede a una página web, el navegador utiliza el método GET para solicitar la página y cualquier recurso adicional (como imágenes, archivos CSS, etc.) que se necesite para mostrar la página correctamente.
- Realizar búsquedas: Los formularios de búsqueda suelen utilizar el método GET para enviar los términos de búsqueda al servidor y recibir los resultados correspondientes.
- Acceder a recursos públicos: Cuando se desea acceder a recursos públicos como imágenes, videos, archivos PDF, etc. que no requieren autenticación o autorización especial.
-

Si los datos son sensibles o requieren protección, se recomienda utilizar el método POST.

Método POST

El método POST es otro de los métodos de solicitud HTTP que se utiliza para enviar información a un servidor web. A diferencia del método GET, el método POST envía los datos como parte del cuerpo de la solicitud HTTP en lugar de incluirlos en la URL.

Cuando se utiliza el método POST, los datos se envían como pares clave-valor en el cuerpo de la solicitud HTTP. Por ejemplo, si tienes un formulario HTML que envía dos parámetros (nombre y apellido) utilizando el método POST, los datos se envían como parte del cuerpo de la solicitud HTTP en lugar de incluirlos en la URL. El cuerpo de la solicitud HTTP podría verse así:

```
nombre=Juan&apellido=Pérez
```

En este ejemplo, nombre es el nombre del primer parámetro y Juan es el valor del primer parámetro, apellido es el nombre del segundo parámetro y Pérez es el valor del segundo parámetro.

Cuando el servidor recibe una petición HTTP utilizando el método POST, los datos se pueden acceder desde el script PHP utilizando la variable global $_POST. Esta variable es un arreglo asociativo que contiene los nombres de los parámetros como claves y los valores de los parámetros como valores. Por ejemplo, para acceder a los valores de los parámetros nombre y apellido en el ejemplo anterior, podrías hacer lo siguiente:

```
$nombre = $_POST['nombre'];
$apellido = $_POST['apellido'];
```

A diferencia del método GET, el método POST no muestra los datos enviados en la URL y, por lo tanto, es más seguro para enviar información confidencial, como contraseñas, información de tarjetas de crédito, etc. Además, el método POST no tiene limitaciones de tamaño para los datos enviados, mientras que el método GET tiene un límite de tamaño (que depende del navegador y el servidor).

Es importante tener en cuenta que, al igual que con el método GET, el método POST no es adecuado para enviar datos sensibles a través de una conexión no segura (HTTP en lugar de HTTPS). Para enviar datos sensibles de manera segura, se recomienda utilizar una conexión HTTPS, que cifra los datos enviados entre el navegador y el servidor.

El método POST se utiliza para enviar información al servidor de manera segura y eficiente, y es especialmente útil para enviar datos sensibles o grandes. Los datos enviados a través del método POST se pueden acceder desde el script PHP utilizando la variable global $_POST.

Ejercitación básica

1. Crear una función que calcule el promedio de un arreglo de números.
2. Crear una función que reciba un número entero y devuelva su factorial.
3. Crear una función que reciba una cadena de texto y devuelva la cantidad de palabras que contiene.
4. Crear una función que reciba un arreglo de números y devuelva la suma de los números pares.
5. Crear una función que reciba un arreglo de números y devuelva el valor máximo.
6. Crear una función que reciba un arreglo de números y devuelva el valor mínimo.
7. Crear una función que reciba un número y devuelva su valor absoluto.
8. Crear una función que reciba dos números y devuelva el resultado de elevar el primero al segundo.
9. Crear una función que reciba un arreglo de números y devuelva un nuevo arreglo con los números ordenados de menor a mayor.
10. Crear una función que reciba un número y devuelva un mensaje indicando si el número es par o impar.

Ejercitación Media

Ejercicio 1:
Escribe una función llamada generarPassword() que genere una contraseña aleatoria de longitud variable. La función debe recibir como parámetro la longitud deseada de la contraseña y devolver la contraseña generada.

Ejercicio 2:
Crea una función llamada validarFecha() que reciba una cadena en formato "dd/mm/aaaa" como parámetro y verifique si la fecha es válida. La función debe devolver true si la fecha es válida o false si no lo es.

Ejercicio 3:
Escribe una función llamada calcularFactorial() que calcule el factorial de un número entero no negativo dado como parámetro. El factorial de un número se define como el producto de todos los números enteros positivos desde 1 hasta el número dado.

Ejercicio 4:
Crea una función llamada ordenarArreglo() que reciba un arreglo numérico como parámetro y lo ordene de forma ascendente. La función debe devolver el arreglo ordenado.

Ejercicio 5:
Escribe una función llamada obtenerPalindromos() que reciba un arreglo de palabras como parámetro y devuelva un nuevo arreglo que contenga solo las palabras que son palíndromos. Un palíndromo es una palabra que se lee igual de izquierda a derecha que de derecha a izquierda.

Estos ejercicios proporcionan desafíos adicionales donde necesitarás aplicar conceptos más avanzados en el uso de funciones y trabajar con problemas más complejos. Al resolver estos ejercicios, mejorarás tus habilidades de programación y tu comprensión de cómo utilizar funciones para resolver problemas específicos.